

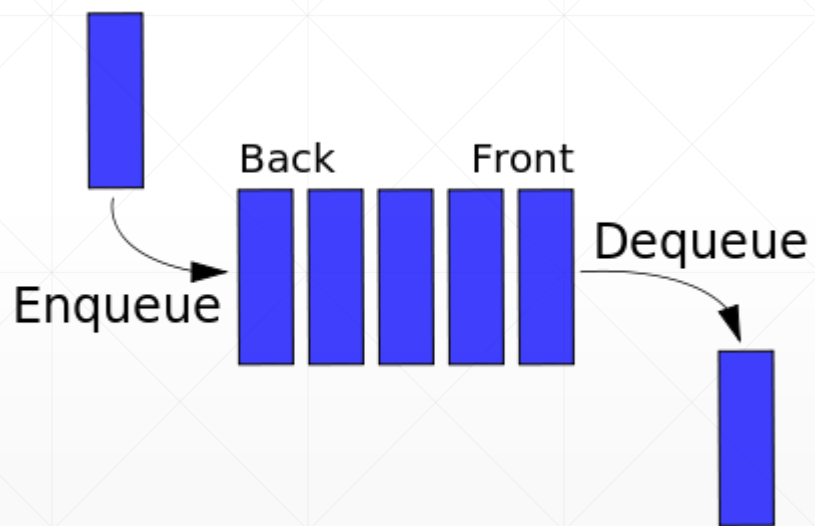


队列

北京理工大学计算机学院
金旭亮

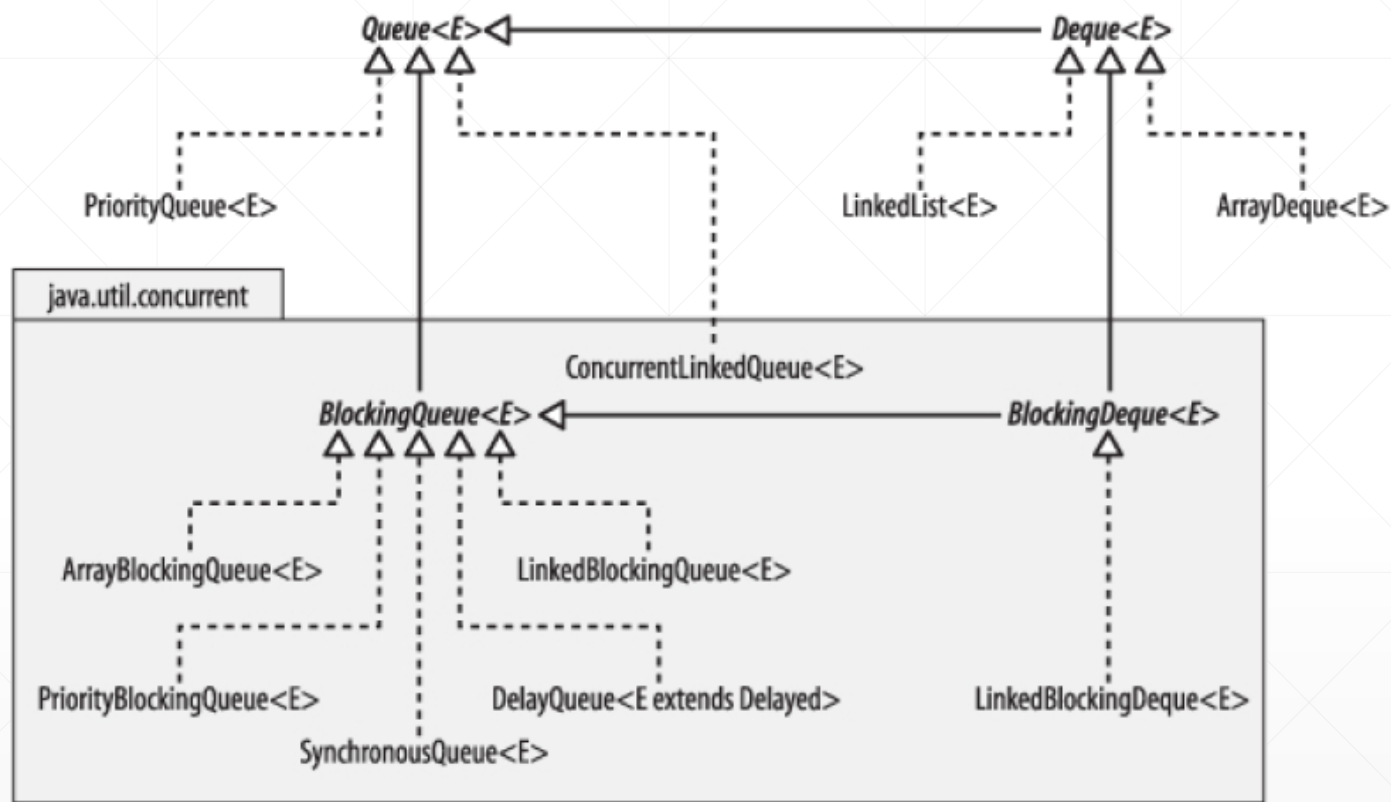
Queue (队列)

队列是一种先入先出 (FIFO: First In First Out) 的数据结构。



基本操作就是两个：
入队 (Enqueue) 和出队 (Dequeue)

JDK所提供的队列相关类型汇总



Queue接口代表的是“经典”的队列，JDK中还扩充定义了另一个Deque接口，代表“双端队列”，用途更广。

队列的基本操作

- JDK 中定义了一个Queue接口，其中定义了以下基本操作：

```
Queue
├── Collection
│   ├── add(E): boolean ↑Collection
│   ├── offer(E): boolean
│   ├── remove(): E
│   ├── poll(): E
│   ├── element(): E
│   └── peek(): E
```

offer：入队，队列满时，返回false。

add：入队，队列满时抛出异常。

element和**peek**：读取而不出队。当队列为空时，前者抛出异常，后者返回null。

remove和**poll**：出队。当队列为空时，前者抛出异常，后者返回null。

队列的基本用法

//ArrayDeque是JDK中常用的队列类型

```
Queue<String> q = new ArrayDeque<>();  
q.add("One");  
q.add("Two");  
q.add("Three");  
q.add("Four");  
System.out.println("原始队列: "+q);
```

示例: UseQueue.java

```
System.out.println("出队两个元素: ");  
System.out.println("出队> " + q.remove());  
System.out.println("出队> " + q.poll());  
System.out.println("队列中剩余元素个数: "+q.size());  
System.out.println(q);  
System.out.println("向队列中批量添加三个元素");  
var strList : List<String> = List.of("Five", "Six", "Seven");  
q.addAll(strList);  
System.out.println(q);  
  
System.out.println("持续出队, 直到队列为空.....");  
while(!q.isEmpty()){  
    System.out.println("item> " + q.remove());  
}  
System.out.println("队列为空时, 再次出队, " +  
    "将会抛出java.util.NoSuchElementException");  
q.remove();
```

优先级队列 (PriorityQueue)



优先级队列是另一种很常见的队列，非线程安全，无容量限制。允许元素重复（**注意这点与后面课程要介绍的TreeSet不同**）。



其顺序由元素自己（如果实现了Comparable接口）或Comparator提供，优先级队列会将它所包容的元素依据大小顺序自动排序。



PriorityQueue总是**按照大小顺序取出元素**，而不是按其插入的先后顺序。

优先级队列示例分析-创建队列

TestPriorityQueue.java示例展示了优先级队列的基本用法:

```
//创建一个测试用的优先级队列
private static Queue<Integer> getExamplePriorityQueue() {
    Queue<Integer> pq = new PriorityQueue<>();
    //下面代码依次向pq中加入6个元素, 其中有重复的
    pq.offer(e: 6);
    pq.offer(e: -3);
    pq.offer(e: 9);
    pq.offer(e: 0);
    pq.offer(e: 110);
    pq.offer(e: 9);
    return pq;
}
```

这里加入的元素, 不仅有重复的, 并且是未经过排序的。

优先级队列示例分析-查看队列内容

```
//查看队列内容而不出队
private static void showPriorityQueueContent() {
    var priorityQueue : Queue<Integer> = getExamplePriorityQueue();
    //foreach迭代访问优先级队列
    //输出: -3,0,9,6,110,9,
    for (var num : priorityQueue) {
        System.out.print(num + ",");
    }
    System.out.println();
    //使用iterator访问优先级队列
    //输出: -3,0,9,6,110,9,
    var iterator : Iterator<Integer> = priorityQueue.iterator();
    while (iterator.hasNext()) {
        System.out.print(iterator.next() + ",");
    }
    System.out.println();
    //最方便的方法, 调用toString()方法得到队列内容
    //输出: [-3, 0, 9, 6, 110, 9]
    System.out.println(priorityQueue);
}
```

使用foreach和iterator都可遍历优先级队列的所有内容, 两者功能是等价的。

最简单的查看优先级队列内容的方式, 是直接调用它的toString()方法。

百家争鸣:

这样的对象集合, 已不具备“先进先出”的队列基本特征, 是否应该还算是一个队列?

优先级队列示例分析-出队

```
//队列中的所有元素，全部出队
private static void pollAll() {
    var priorityQueue : Queue<Integer> = getExamplePriorityQueue();
    //依次出队列，可以看到，并不是按元素的加入顺序排列，
    //而是按元素的大小顺序出队的
    //输出: -3,0,6,9,9,110,
    while (!priorityQueue.isEmpty()) {
        System.out.print(priorityQueue.poll() + ",");
    }
    System.out.println();
    //现在队列为空，输出: []
    System.out.println(priorityQueue);
}
```

出队时，严格按照元素大小顺序出队。

要求队列中元素要不自己实现Comparable接口，要不提供一个大小比较器，以确定两个元素的大小。

优先级队列示例分析-常规操作

```
private static void peekPollAndRemove() {  
    var priorityQueue : Queue<Integer> = getExamplePriorityQueue();  
    System.out.println("队列中原有成员: ");  
    System.out.println(priorityQueue);  
    //访问队列第一个元素(不出队) , 其实就是队列中最小的元素: -3  
    System.out.println("\n查询最小元素 (在队头) : "+priorityQueue.peek());  
    //从队首移除一个元素  
    priorityQueue.poll();  
    System.out.println("\n移除队列中的head元素之后");  
    System.out.println(priorityQueue);  
    //查找并移除元素9 (由于有重复的, 仅移除第一个)  
    priorityQueue.remove(o: 9);  
    System.out.println("\n 移除队列中的元素9之后");  
    System.out.println(priorityQueue);  
    //又加入一个新元素56  
    priorityQueue.offer(e: 56);  
    System.out.println("\n 向队列中添加元素56之后");  
    System.out.println(priorityQueue);  
}
```

Run: TestPriorityQueue x

"C:\Program Files\Java\jdk-15\bin\java.exe"

队列中原有成员:
[-3, 0, 9, 6, 110, 9]

查询最小元素 (在队头) : -3

移除队列中的head元素之后
[0, 6, 9, 9, 110]

移除队列中的元素9之后
[0, 6, 110, 9]

向队列中添加元素56之后
[0, 6, 110, 9, 56]

Process finished with exit code 0

优先级队列用在哪？



到银行办理业务，分为“个人业务”和“对公业务”两类。“对公业务”优先级高于“个人业务”。

银行采用了一个“取号排队”方案以改进服务质量。

客户随机到来，进入等待队列，“个人业务”按照到达时间依次办理，如有“对公业务”，在办完当前个人业务之后，选择“对公业务”而不是队列中原先排序的“下一个”个人业务。

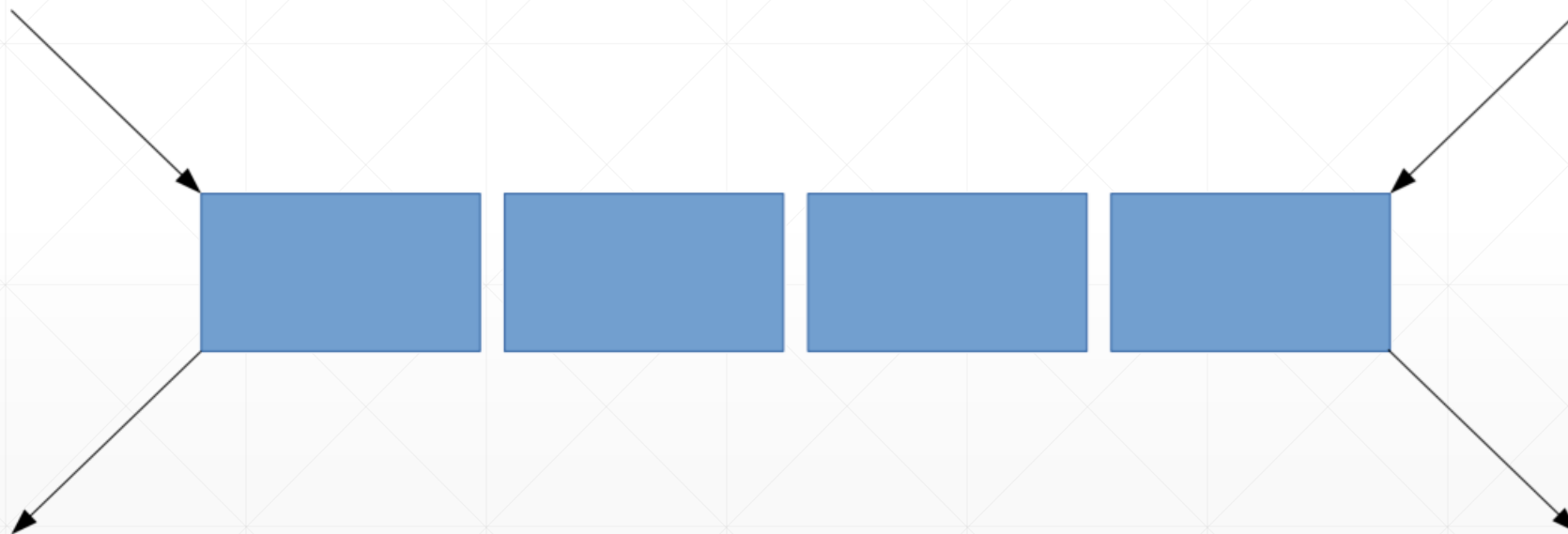
夯实基础：PriorityQueues的内部数据结构



- JDK中的PriorityQueue在内部使用“priority heaps”，这是一种特殊的二叉树，拥有以下特性：
 - 父节点一定小于子节点
 - 除了最底层的节点，其它节点都必须是“满树”，就是说不允许中间节点中出现只有一个子节点的情况，不满足这个条件时整个树需要重构。
 - 当向heap中追加元素时，heap会自动进行调整
- 《数据结构》教材中，通常都会有相应的章节介绍优先队列与堆，感兴趣的同学可以找本教材自学这块的内容，然后再阅读JDK中的示例源码。

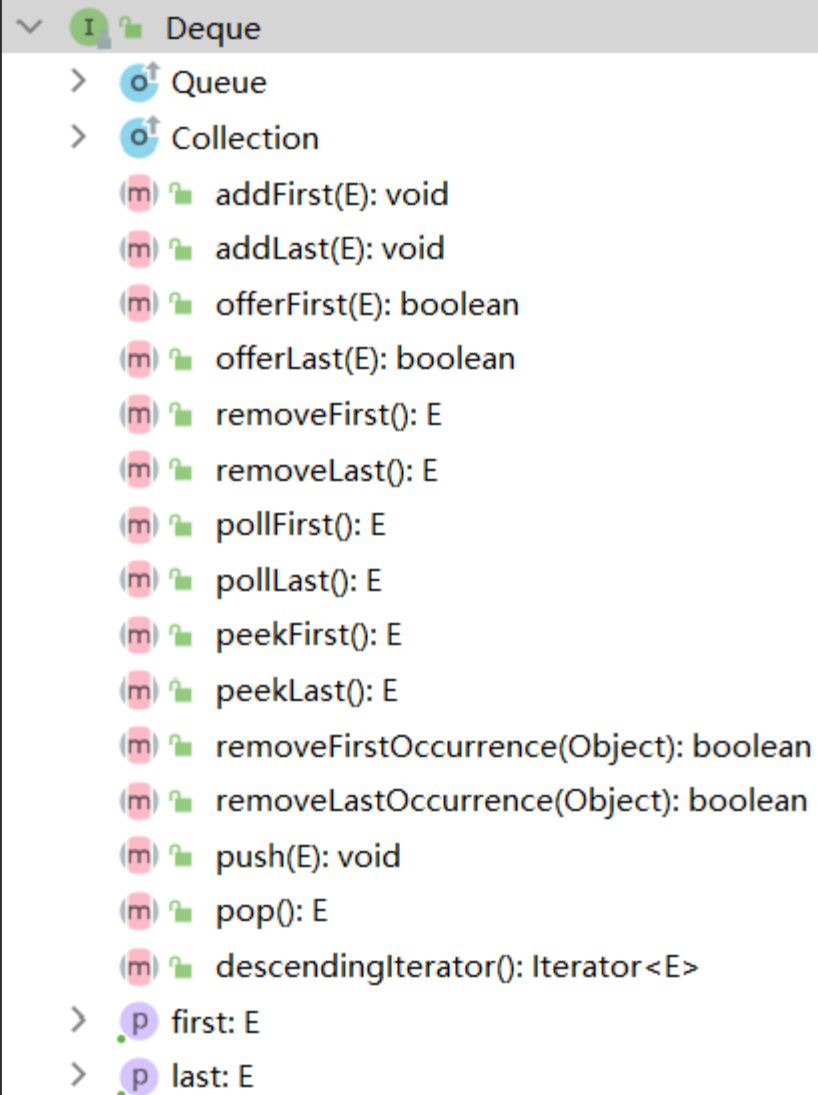
双端队列 (Deque)

Deque = Double Ended Queue



在两端都可以入队和出队

功能强大的Deque接口



```
Deque
> Queue
> Collection
  (m) addFirst(E): void
  (m) addLast(E): void
  (m) offerFirst(E): boolean
  (m) offerLast(E): boolean
  (m) removeFirst(): E
  (m) removeLast(): E
  (m) pollFirst(): E
  (m) pollLast(): E
  (m) peekFirst(): E
  (m) peekLast(): E
  (m) removeFirstOccurrence(Object): boolean
  (m) removeLastOccurrence(Object): boolean
  (m) push(E): void
  (m) pop(): E
  (m) descendingIterator(): Iterator<E>
> .p first: E
> .p last: E
```

- JDK中的Deque接口用于定义双端队列，这种队列，既象Queue，又象Stack。

实现Deque接口的两个常用类型是ArrayDeque和LinkedList。

双端队列—添加元素

```
boolean offerFirst(E e)  
boolean offerLast(E e)
```

```
void addFirst(E e)  
void addLast(E e)
```

- offer*: 当队列满时，返回null
- add*: 当队列满时，抛出异常

双端队列—移除元素

```
E removeFirst()
```

```
E removeLast()
```

```
E pollFirst()
```

```
E pollLast()
```

当队列为空时，remove*抛出异常，poll*返回null

双端队列—读取而不移除

```
E getFirst()
```

```
E getLast()
```

```
E peekFirst()
```

```
E peekLast()
```

当队列为空时，get*抛出异常，peek*返回null

把双端队列当成堆栈用.....

```
void push(E e)
```

```
E pop()
```

上述方法的功能与普通的堆栈一致。



像JDK的这种设计，在一个接口中集成多种职责，引发了不少争议，不少人认为这虽然最大限度地重用了代码，但违背了常识和基本的面向对象设计原则，并非一种好的设计方案。

ArrayDeque作为堆栈用

ArrayDeque是最常用的集合类型之一，它既可以当作队列使用，也可以当成堆栈使用，前面介绍过了，这里再复习一下：

```
public class DequeueAsStack {  
    public static void main(String[] args) {  
        var original : List<String> = Arrays.asList("One", "Two", "Three");  
        Deque<String> stack = new ArrayDeque<>();  
        //从List中取出所有的元素, push到堆栈中  
        for (String s : original) {  
            stack.push(s);  
        }  
        //不断地出栈, 输出: Three|Two|One|  
        while (stack.size() > 0) {  
            System.out.print(stack.pop()+"|");  
        }  
    }  
}
```

自主学习



前面介绍过的LinkedList，也实现了Deque接口，因此，也可以作为双端队列和栈使用。

请编写示例代码，使用LinkedList作为双端队列和栈。